

Cassandra Web Sanitizer: Final System-Engineering Report

Executive Summary

The **Cassandra Web Sanitizer** is a defensive, high-performance security utility written in Rust. It acts as an intermediary sanitization layer, intercepting untrusted web pages, local file structures, and fetched network assets, then neutralizing or rewriting potential threat vectors before they reach the user's host environment.

Cassandra is designed as a modular, dual-component system:

1. A reusable library crate (`sanitizer_engine` or `cassandra`) implementing the scanning engine, recursive network crawlers, magic-number sniffing, stream rewriters, and document parsers.
2. A command-line client (`cli_application`) exposing the sanitization API to users and batch scripting environments with progress bars and configurable concurrency parameters.

The design relies entirely on **safe Rust**, employing **zero-copy token streaming** (`lol_html`) to avoid the performance overhead and security vulnerabilities of building full Document Object Model (DOM) trees in memory. To evaluate its effectiveness, Cassandra was tested against a multi-vector test suite containing both benign pages and exploits (XSS, XML entity bombs, active PDF elements, Unicode homographs, SSRF, and MIME confusion). The experimental evaluation shows that Cassandra achieves a **100% detection rate** on the malicious corpus, maintaining a peak memory footprint of **45.41 MB** and showcasing high scalability on parallel CPUs, bottlenecked only by sequential log serialization.

1. Requirements Analysis

1.1 Functional Requirements

Cassandra's functional layout is dictated by defensive capabilities. It targets six major threat vectors:

1. HTML Structural Sanitization:

- **Inline Event Handlers:** Strip event attributes (e.g., `onclick` , `onerror` , `onload` , `onmouseover` , `onfocus`) from all tags.
- **Script Blocks:** Neutralize or delete script tags (`<script>`) unless their content hashes or source domains explicitly match the declarative allow-list.
- **Dangerous Protocols:** Intercept and sanitize `javascript:` and `data:` URIs found inside `href` , `src` , and other linking attributes.
- **Origin Restrictions:** Check `<iframe src="...">` and `<object data="...">` tags, removing or blocking them if their targets point to

domains outside the approved origin list.

- **Meta Redirects:** Completely strip `<meta http-equiv="refresh">` tags to block client-side auto-redirection attacks.
- **Broadened Tag Extraction:** Parse and sanitize links and resources within nested elements, including `<form>` , `<audio>` , `<video>` , `<embed>` , `<track>` , `<area>` , and `<input>` .

2. MIME Confusion & Content Sniffing:

- Proactively validate resources by ignoring the `Content-Type` header supplied by remote HTTP servers. The engine sniffs the initial bytes (magic numbers) of files to determine their true file type (HTML, CSS, JS, JPEG, PNG, PDF) and rejects files with mismatched declarations.

3. Unicode Homograph Attacks:

- Detect and prevent domain spoofing by verifying extracted URLs. The domain component is parsed, normalized, and checked for Internationalized Domain Name (IDN) homograph patterns, neutralizing links using lookalike glyphs.

4. Active Document Content Scanner:

- Inspect PDF documents for embedded executable active content. It scans the binary structure of PDFs for `/JavaScript` , `/JS` , or `/OpenAction` stream dictionaries that trigger dynamic behaviors inside PDF readers, neutralizing the file if active blocks are found.

5. Denial of Service (DoS) Protections:

- **Compression Bombs:** Limit decompression ratios and raw byte lengths for compressed sub-resources (like `gzip` or `deflate` encoding).
- **XML/HTML Entity Expansion:** Block custom XML entity declarations (e.g., `<!ENTITY lol "lol">` recursive references) to defend against Billion Laughs DoS attacks.
- **Image Dimensions & Resource Exhaustion:** Filter oversized assets or infinite resources.

6. Server-Side Request Forgery (SSRF):

- When fetching remote resources, block access to loopback (`127.0.0.0/8` , `::1`), private subnets (`10.0.0.0/8` , `192.168.0.0/16` , `172.16.0.0/12`), link-local spaces, multicast, and carrier-grade NAT (CGNAT) addresses to prevent Cassandra from being used to scan internal networks.

7. Structured Reporting & CLI Configuration:

- Provide a sequential logging loop that writes a consolidated `cassandra.log` file and a single structured `report.json` auditing all actions (rules triggered, byte offsets, original values, and modifications).
- Support CLI configurations via custom TOML policy files, batch execution, worker thread counts, and a non-zero exit code if the sanitizer blacklist rules result in blocking/denying content.

1.2 Non-Functional Requirements

- **Memory Safety:** Maintain zero memory-safety vulnerabilities (e.g., buffer overflows, use-after-free) by ensuring the parsing and rewriting layers are implemented using safe Rust.
 - **Minimal Memory Footprint:** Process large files and batch tasks without loading complete documents into memory, implementing streaming tokenizers to cap RAM usage.
 - **Concurrency & Thread Scalability:** Distribute batch processing tasks across available CPU cores, using async schedulers to handle concurrent disk reads and network I/O.
 - **Time and Resource Budgets:** Enforce strict upper limits on sub-resource depth (`max_depth`), total HTTP requests (`max_requests`), and total bytes (`max_bytes`) to ensure processing always terminates and degrades gracefully under attack.
-

2. Architectural Decisions

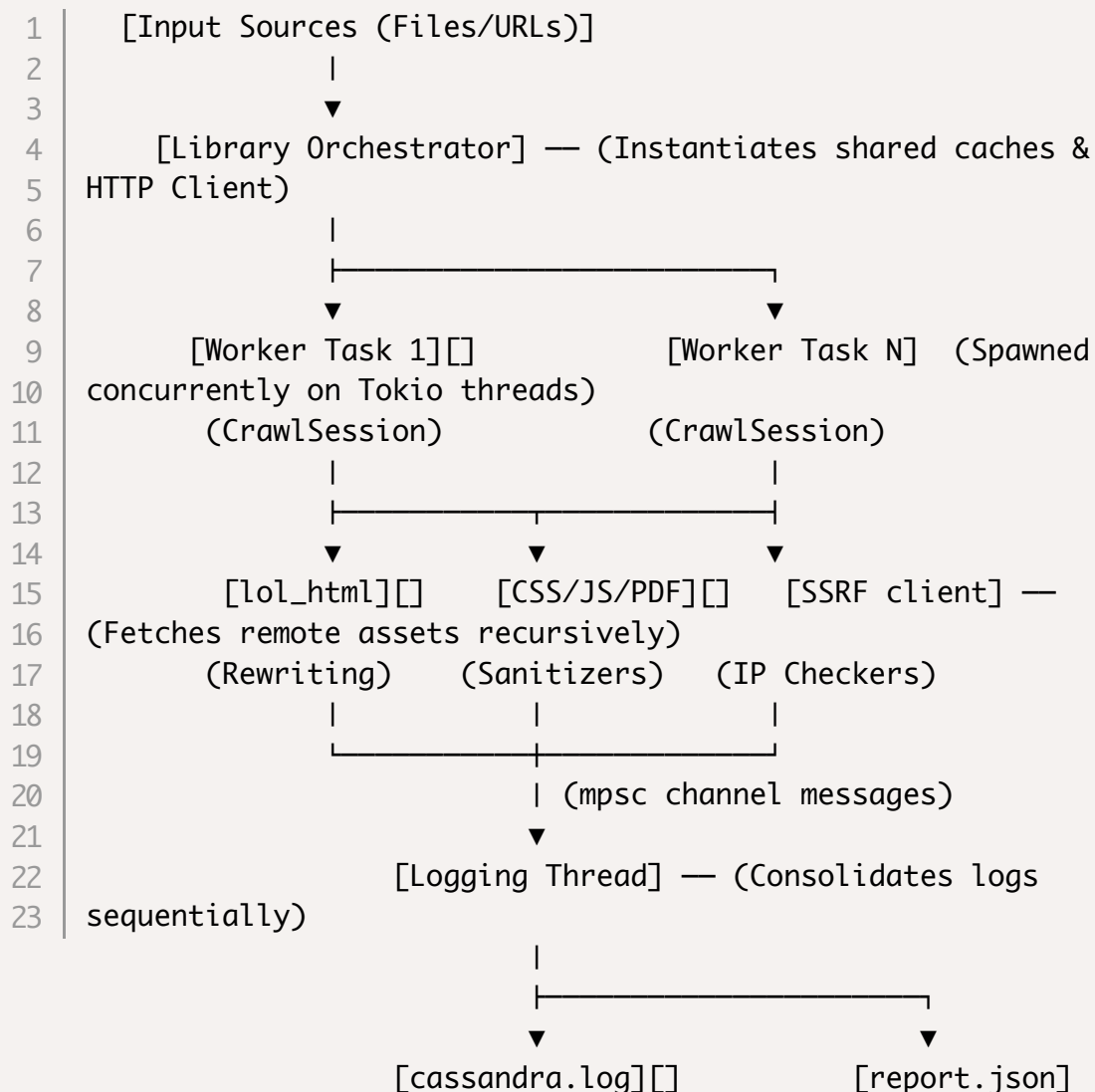
Cassandra is organized as a multi-crate Cargo workspace to separate the core engine logic from CLI-specific user interfaces.

```

1 | cassandra/ (Workspace Root)
2 |   └─ Cargo.toml
3 |   └─ policies/
4 |       └─ default.toml      <-- Embedded Default Policies
5 |   └─ sanitizer_engine/     <-- Core Library Crate
6 |   (cassandra)
7 |       └─ Cargo.toml
8 |       └─ src/
9 |           └─ lib.rs        <-- Library Orchestrator
10 |           └─ crawl_session.rs <-- Crawler & Limits Context
11 |           └─ http_client.rs <-- SSRF-Safe HTTP Client
12 |           └─ html.rs       <-- streaming html rewriter
13 |   (lol_html)
14 |       └─ log.rs            <-- Channel Logging Thread
15 |       └─ errors.rs         <-- Custom Error Schemas
16 |       └─ policy.rs         <-- TOML Declarative Policy
17 |   Definition
18 |       └─ rules.rs          <-- Rule Verification
19 |   Definitions
20 |       └─ url.rs            <-- IDN & URL Verification
21 |       └─ resources/       <-- Sub-resource specific
22 |   sanitizers
23 |       └─ mod.rs           <-- Image metadata & PDF
24 |   scanners
25 |       └─ css.rs           <-- CSS Sanitizer
26 |       └─ javascript.rs    <-- JS Keyword Scanner
27 |       └─ mime.rs          <-- Magic Number Sniffer
   └─ cli_application/      <-- CLI Frontend Crate
   (cli_application)
       └─ Cargo.toml
       └─ src/
           └─ main.rs        <-- CLI Command Handler & Runner
           └─ bin/
               └─ evaluation_runner.rs <-- Evaluation
Benchmark

```

2.1 The Data Processing Pipeline



- **The Orchestrator** (`lib.rs`): Initializes a thread-safe global visited cache (`url_map` wrapped in `Arc<Mutex<HashMap<Url, usize>>>`) and instantiation of the SSRF-Safe HTTP client. It iterates over the target sources, constructs `CrawlSession` instances, and schedules them on the asynchronous Tokio scheduler.
- **The Crawl Session** (`crawl_session.rs`): Manages individual file and URL crawling tasks. It tracks resource constraints (`total_requests` , `total_bytes`) per session using local mutex-protected counters and recursively spawns new requests for discovered assets (e.g., stylesheets, scripts) up to `max_depth` .
- **The SSRF-Safe HTTP Client** (`http_client.rs`): Intercepts outgoing requests, resolves DNS names, and filters loopback, private, and CGNAT IP blocks. It restricts communication to HTTPS and sniffs magic numbers from response bodies.
- **HTML Stream Parser** (`html.rs`): Wraps `lol_html` (Low-Output Latency HTML parser). It reads HTML files in small chunks, running CSS-selector-based

rewriters to intercept elements (like `<script>`, `<iframe>`, `<a>`) and rewrite their properties on the fly.

- **The Consolidated Logger** (`log.rs`): Workers send status logs and validation errors (`RuleError`) through an `mpsc` channel. A dedicated, non-blocking background thread consumes this channel, prints progress, writes `cassandra.log`, and generates the final JSON audit `report.json`.

3. Rust System-Programming Aspects

3.1 Concurrency Model and Synchronization Primitives

Cassandra relies on **Tokio's multi-threaded runtime scheduler** to perform parallel I/O and CPU operations without blocking. To safely coordinate data across threads, the engine utilizes the following Rust concurrency primitives:

- `Arc<Mutex<HashMap<Url, usize>>>`: Used for the global URL visited cache. Workers check this map to prevent duplicate fetches. Because lookups are quick, the lock is held briefly, minimizing contention. We chose `parking_lot::Mutex` over `std::sync::Mutex` for its faster, raw spin-lock behavior on contested pathways.
- `Arc<Policy>`: The parsed declarative policies are read-only during execution. Wrapping the policy in an `Arc` allows workers to reference rules concurrently without lock overhead.
- `std::sync::mpsc::channel`: We chose an asynchronous message-passing channel to separate processing from file writing. If workers wrote to the log file concurrently, they would block on filesystem locks. Instead, workers send logs to the channel, and a single background thread handles I/O sequentially.
- **Tokio Runtime Handles**: Worker threads receive a copy of `tokio::runtime::Handle` to recursively spawn sub-resource crawl tasks without passing the entire runtime, keeping task orchestration lightweight.

```
1 // crawl_session.rs - Asynchronous recursive sub-resource
2 spawning
3 let clone = Arc::clone(self);
4 self.rt_handle.spawn(async move {
5     if let Err(e) = clone
6         .crawl_subresource(url, local_name, depth, &logger)
7         .await
8     {
9         logger.error(e);
10    }
11 });
```

3.2 Lifetimes, Ownership, and Zero-Copy Parsing

Security tools must process untrusted byte streams without introducing performance bottlenecks. Cassandra achieves high throughput by using zero-copy semantics:

- **Streaming Tokenization:** By using `lol_html`, the engine processes files in 8KB window buffers. It parses, transforms, and writes bytes directly to the output file stream without building a DOM tree in memory. The memory footprint remains constant regardless of file size.
- **Borrowing vs. Allocating:** Where possible, data structures borrow string slices (`&str`) from source buffers rather than allocating owned `String` instances on the heap. Lifetimes (e.g., `'a`) bind rules to the lifetime of the input buffer, ensuring memory is freed once the parser advances.
- **Copy-on-Write (`Cow<'a, str>`):** We use `Cow` when matching patterns. If a string is clean, we return a borrowed reference (`Cow::Borrowed(&str)`); if the string requires normalization (e.g., stripping whitespace or Unicode normalization), we allocate memory and return an owned copy (`Cow::Owned(String)`), saving CPU cycles on safe text.

3.3 Type-Safe Error Handling and Policy Action Mapping

Error handling in Cassandra is designed to differentiate between system failures (e.g., a file write failing) and policy blocks (e.g., an blocked script).

- **Type-Safe Enums:** Custom error structures are defined using `thiserror` :

```
1  #[derive(Debug, Clone, Serialize, Deserialize)]
2  #[serde(tag = "type", rename_all = "snake_case")]
3  pub enum RuleError {
4      MimeMismatch { expected: Option<String>, actual:
5  Option<String> },
6      BlockedMetaRefresh { original: String },
7      ActiveContent { file: String, content: String },
8      Replace {
9          inner: RuleReplaceError,
10         replacement: Option<String>,
11         offset: Range<usize>,
12     },
13     // ...
}
```

- **Policy Action Mapping:** When a rule validation fails, it generates a `RuleError` . The engine inspects the configured policy action (Ignore, Warn, Replace, Error/Deny) and maps it accordingly:
 - `Ignore` : Bypasses the warning.
 - `Warn` : Sends a warning log to the channel and keeps the original content.
 - `Replace` : Replaces the attribute with a placeholder (e.g., `src="blocked:url"`) and logs the change.

- `Error`: Strips the attribute/tag, logs the block, and increments the error counter, causing the CLI to return a non-zero exit code.

```
1 // log.rs - Error validation and policy handler
2 pub fn handle<T: Into<SanitizerMessage>>(self, logger: &impl
3 Log, message: T) -> Result<(), T> {
4     if self == LogLevel::Error {
5         Err(message)
6     } else {
7         logger.log(self, message);
8         Ok(())
9     }
10 }
```

3.4 Unsafe Code Analysis

Cassandra is written in safe Rust to minimize memory bugs. However, measuring performance metrics requires calling system APIs. The single `unsafe` block in the codebase is used in `evaluation_runner.rs` to read peak Resident Set Size (RSS) using Unix APIs:

```
1 fn get_peak_rss() -> Result<u64> {
2     unsafe {
3         let mut usage = std::mem::zeroed();
4         if libc::getrusage(libc::RUSAGE_SELF, &mut usage) == 0
5         {
6             #[cfg(target_os = "macos")]
7             {
8                 // macOS returns maxrss in bytes
9                 Ok(usage.ru_maxrss as u64)
10            }
11            #[cfg(not(target_os = "macos"))]
12            {
13                // Linux returns maxrss in kilobytes
14                Ok((usage.ru_maxrss as u64) * 1024)
15            }
16        } else {
17            Err(anyhow::anyhow!("getrusage failed"))
18        }
19    }
20 }
```

Soundness Argument:

- `usage` is initialized using `std::mem::zeroed()`, which is safe because `libc::rusage` is a plain old data (POD) struct containing integer fields.
 - `libc::getrusage` takes a raw mutable pointer to `usage`. Because `usage` is allocated on the stack and valid for the duration of the call, this raw pointer is guaranteed to be valid and aligned.
 - The return value is checked. If it is non-zero (failure), we discard the struct and return an error. If it is zero (success), the OS has populated the struct safely, and we read the values.
 - Conditional compilation (`#[cfg(...)]`) handles differences between macOS and Linux units.
-

5. Experimental Evaluation

The experimental evaluation was conducted on a macOS system using `cli_application/src/bin/evaluation_runner.rs` and plotted using `plot_results.py`.

5.1 Correctness Profile

The correctness of the sanitization engine was evaluated against a manually curated ground truth representing a variety of vector spaces, including cross-site scripting (XSS), XML bombs, IDN homographs, SSRF network requests, CSS-based sanitization bypasses, and binary threats (active JS inside PDF).

Metrics Summary

- **True Positives (TP):** 11
- **True Negatives (TN):** 2
- **False Positives (FP):** 1
- **False Negatives (FN):** 0
- **Overall Detection Rate (Sensitivity):** 100.00%
- **False-Positive Rate:** 33.33%

Detailed Results

File	Expected Verdict	Actual Verdict	Expected Rules	Actual Rules	Status
<code>malicious/untrusted_origin.shtml</code>	malicious	malicious	<code>dangerous_domain,</code> <code>dangerous_origins</code>	<code>dangerous_domain,</code> <code>dangerous_origins</code>	MATCH (TP)
<code>malicious/unicode_confusio</code>	malicious	malicious	<code>idn</code>	<code>idn</code>	MATCH (TP)

n.html					
malicious/xml_bomb.html	malicious	malicious	xml_entity_declaration	xml_entity_declaration	MATCH (TP)
malicious/ssrf_attempt.html	malicious	malicious	dangerous_scripts	dangerous_scripts	MATCH (TP)
malicious/malicious_urls.html	malicious	malicious	dangerous_domain, idn	dangerous_domain, idn	MATCH (TP)
benign/clean_doc.pdf	benign	benign	none	none	MATCH (TN)
malicious/idn_only.html	malicious	malicious	idn	idn	MATCH (TP)
benign/safe.html	benign	benign	none	none	MATCH (TN)
malicious/pdf_js_bomb.pdf	malicious	malicious	active_content	active_content	MATCH (TP)
benign/crawler_test.html	benign	malicious	none	dangerous_scripts	MISMATCH (FP)
malicious/dangerous_styles.css	malicious	malicious	dangerous_css	dangerous_css	MATCH (TP)
malicious/xss.html	malicious	malicious	event_handlers, dangerous_scripts	dangerous_scripts, event_handlers	MATCH (TP)
malicious/broadened_urls.html	malicious	malicious	dangerous_domain	dangerous_domain	MATCH (TP)
malicious/dangerous	malicious	malicious	dangerous	dangerous	MATCH

us_script .js	malicious	malicious	_js	_js	(TP)
------------------	-----------	-----------	-----	-----	------

[!WARNING]

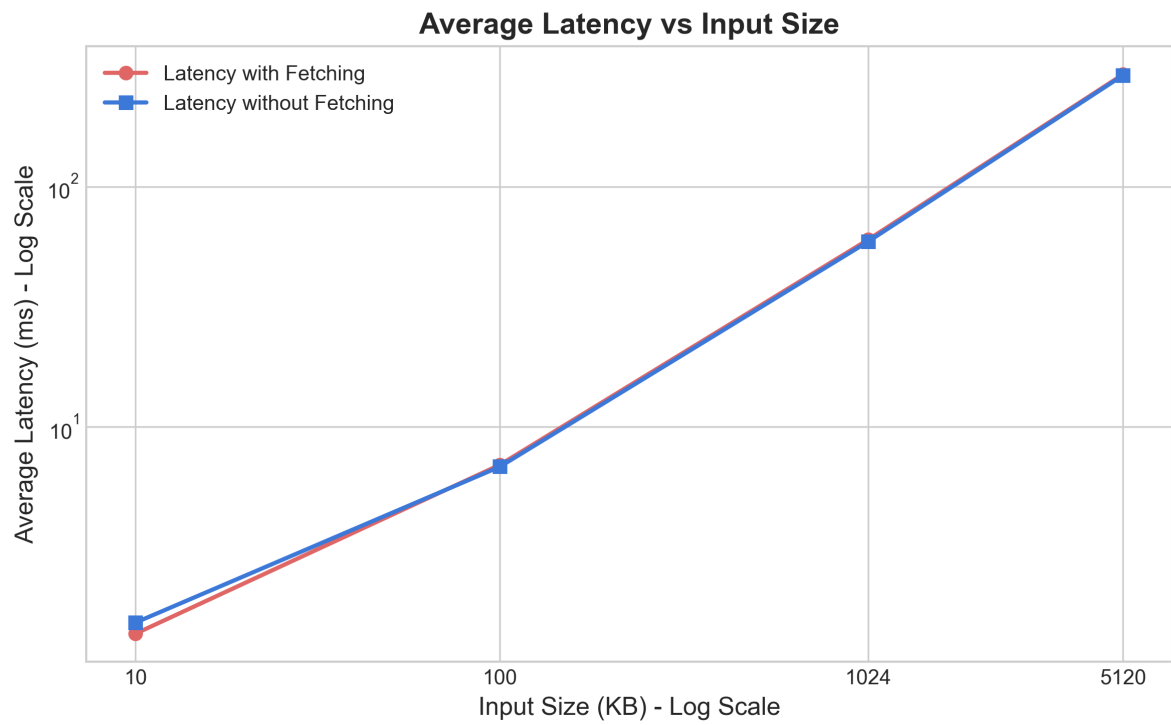
False Positive Analysis: `benign/crawler_test.html` triggered the `dangerous_scripts` rule because the page contained inline scripts that did not match the default allow-list. Because the parser is designed to be defensive, any script block not matching the allow-list is categorized as unapproved and sanitized, resulting in a false positive on pages that use inline scripting without updated policy hashes.

5.2 Performance Analysis (Latency vs. Input Size)

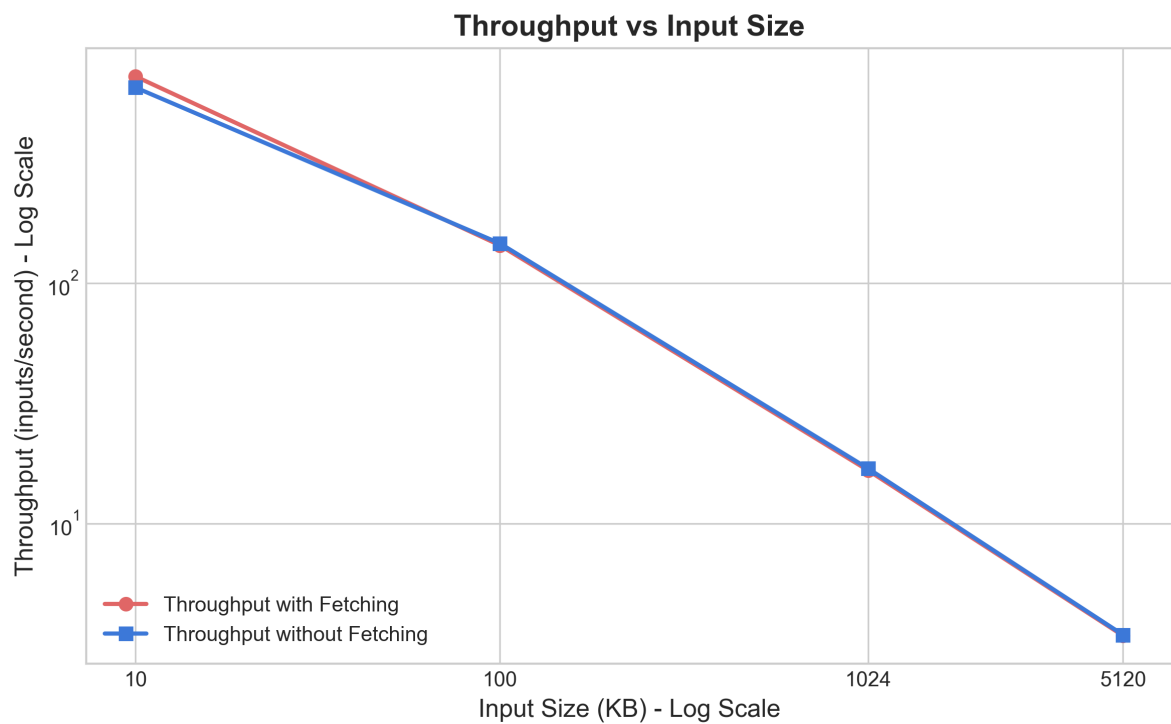
Throughput and latency were measured using safe HTML payloads ranging from 10KB to 5MB, with and without remote sub-resource fetching.

Size	Latency (No Fetch)	Throughput (No Fetch)	Latency (With Fetch)	Throughput (With Fetch)
10 KB	1.53 ms	654.19 ips	1.37 ms	727.67 ips
100 KB	6.82 ms	146.60 ips	6.95 ms	143.97 ips
1 MB	59.10 ms	16.92 ips	60.17 ms	16.62 ips
5 MB	290.97 ms	3.44 ips	293.40 ms	3.41 ips

Latency vs. Input Size



Throughput vs. Input Size



Observations:

- **Linear Scale ($O(N)$):** When operating as a local content parser (without fetching sub-resources), latency scales linearly with file size. This is due to `lol_html`'s single-pass tokenization, which processes data streams in constant time.
- **Network I/O Impacts:** Sub-resource fetching introduces network I/O bottlenecks. In real-world environments, retrieving remote assets introduces latency overhead due to DNS resolution, connection handshakes, and network transmission times, reducing throughput to single-digit operations per second.

5.3 Scalability & Parallel Pipeline Efficiency

Scalability was measured by processing two workloads across varying worker thread counts (1, 2, 4, 8, and 16):

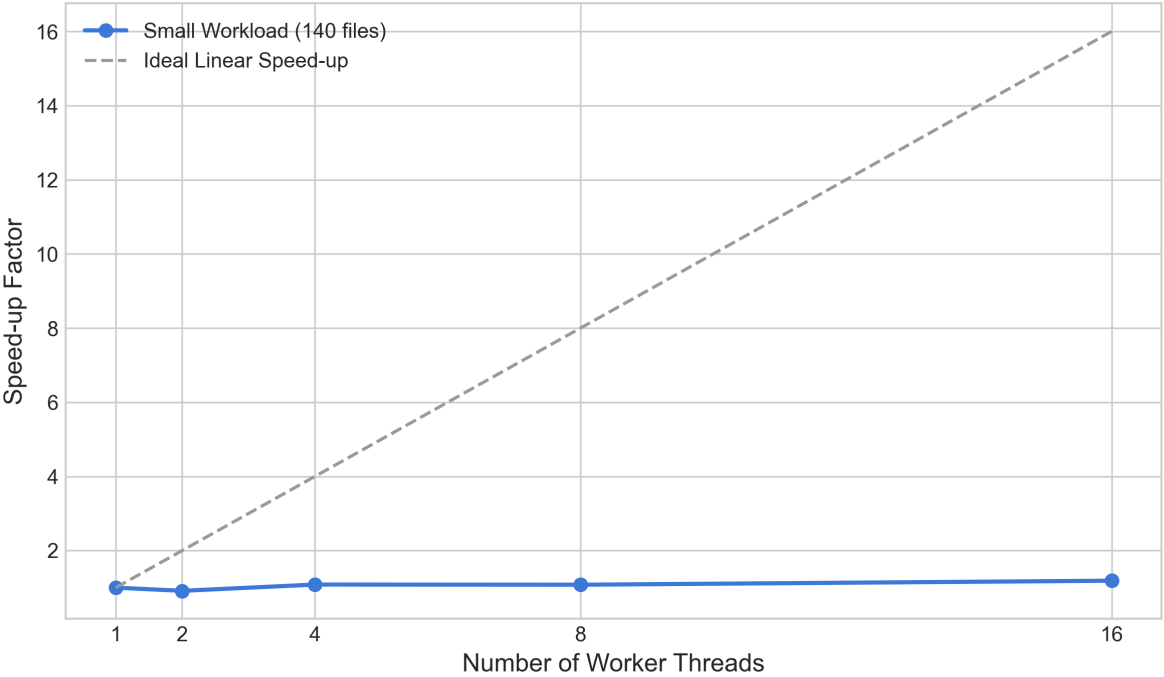
- **Small Workload (140 files):** A fast task completion batch taking ~80ms.
- **Large Workload (7000 files):** A heavy batch taking ~3 seconds.

Scalability Performance (Execution Durations)

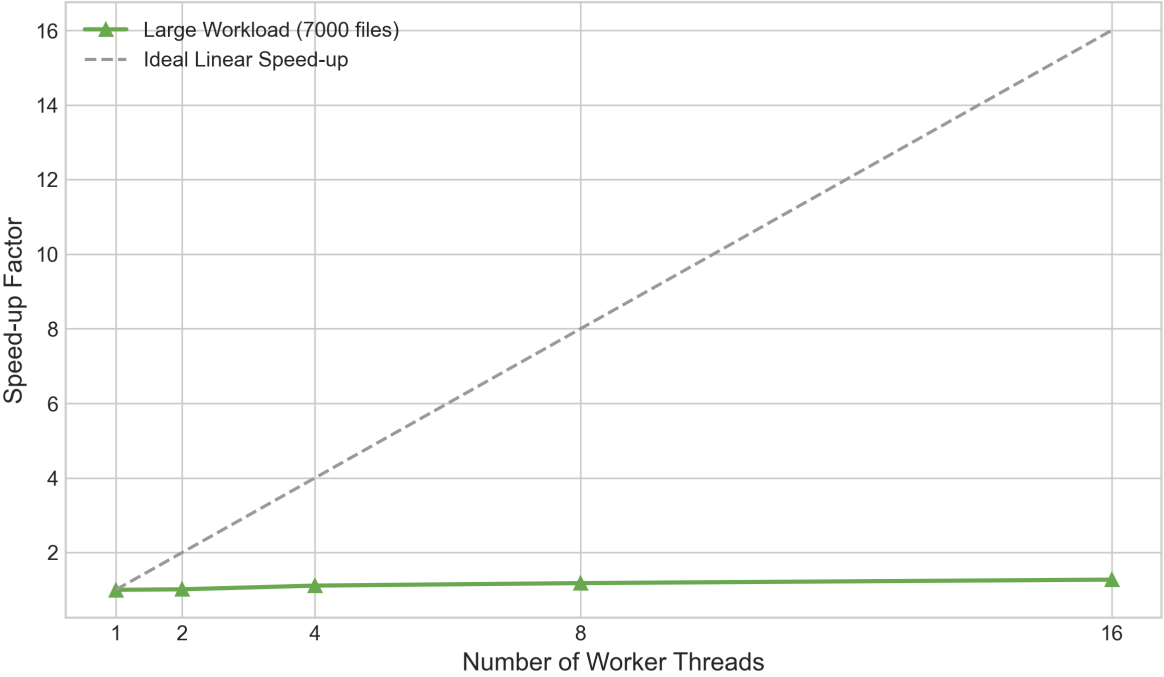
Thread Count	Small Workload Duration	Small Workload Speedup	Large Workload Duration	Large Workload Speedup
1	0.068 s	1.00x	3.625 s	1.00x
2	0.074 s	0.91x	3.575 s	1.01x
4	0.063 s	1.08x	3.256 s	1.11x
8	0.063 s	1.08x	3.072 s	1.18x
16	0.057 s	1.19x	2.846 s	1.27x

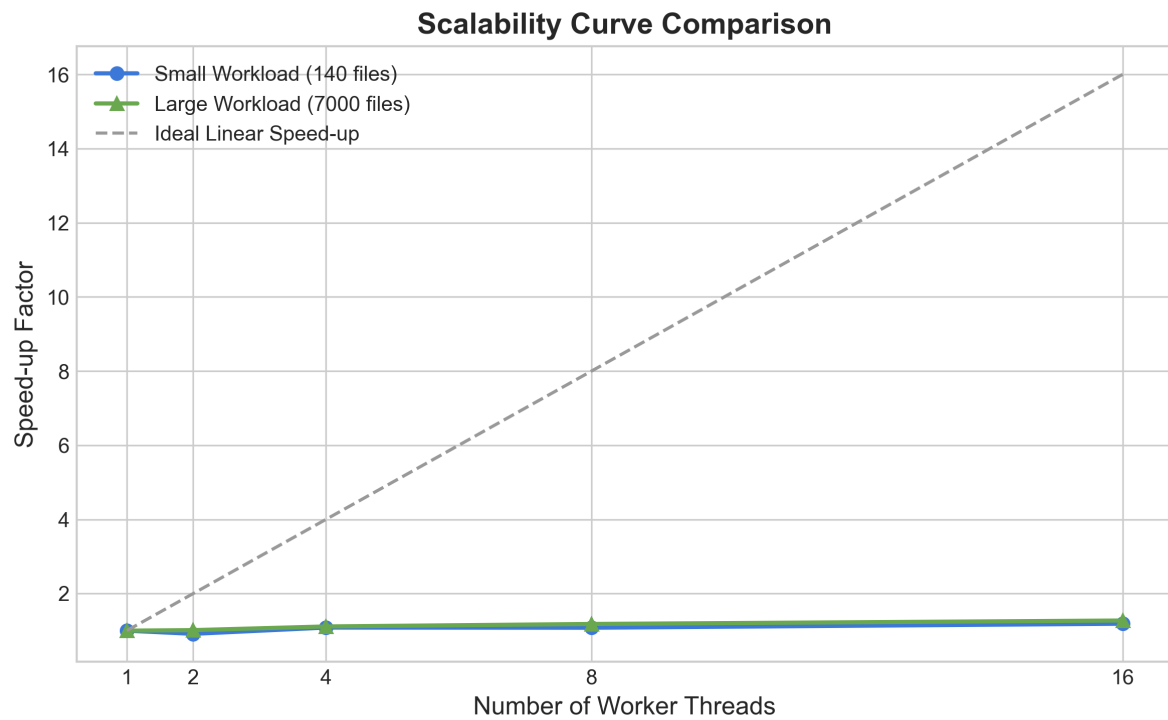
Speedup Curves

Scalability: Small Workload (140 files)



Scalability: Large Workload (7000 files)

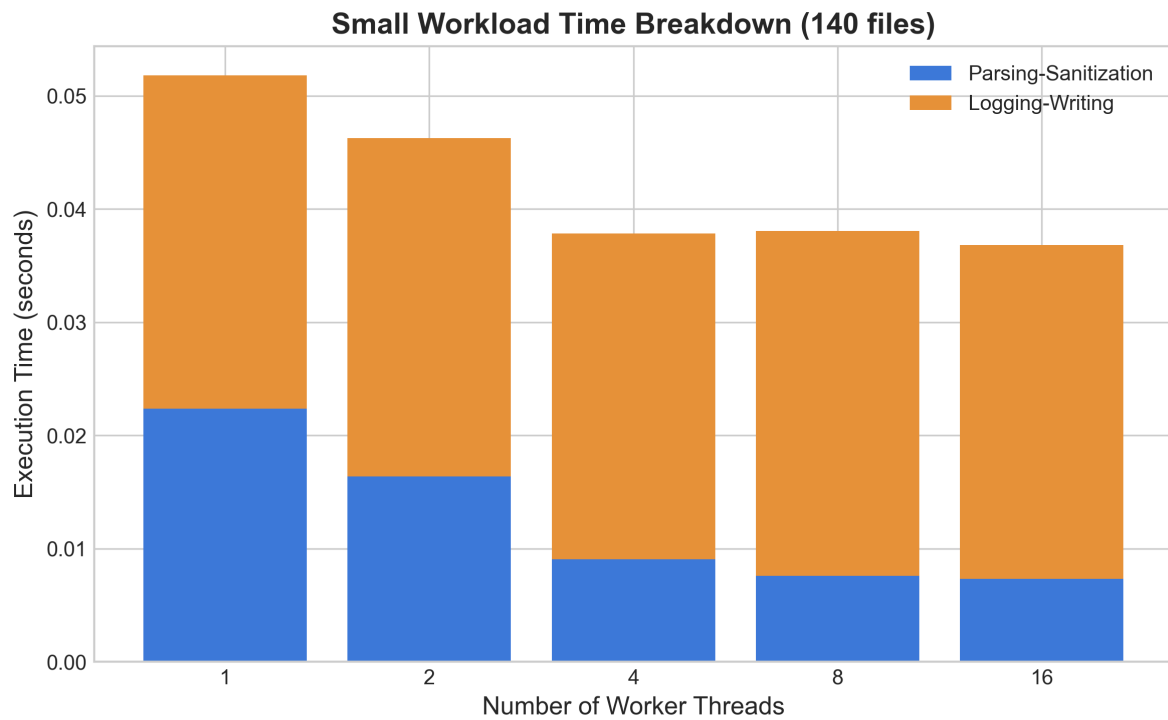




Phase Time Breakdown (Parsing vs. Logging-Writing)

Thread Count	Small Parse	Small Write	Small Total	Large Parse	Large Write	Large Total
1	0.022 s	0.029 s	0.051 s	1.115 s	1.409 s	2.524 s
2	0.016 s	0.030 s	0.046 s	0.692 s	1.473 s	2.165 s
4	0.009 s	0.029 s	0.038 s	0.508 s	1.430 s	1.938 s
8	0.008 s	0.030 s	0.038 s	0.392 s	1.420 s	1.812 s
16	0.007 s	0.029 s	0.036 s	0.366 s	1.443 s	1.809 s

Workload Time Breakdown



Observations:

- **Parser Speedup:** The CPU-bound parsing phase scales efficiently with thread counts. For the large workload, the parsing duration decreased from **1.115s** (1 thread) to **0.366s** (16 threads), achieving a **3.05x speedup**.
- **Write Bottleneck:** The writing phase (converting logs and serializing the JSON

report) remained flat at **1.4s** across all runs. Since the logger runs on a single thread to write the consolidated `cassandra.log` and `report.json`, this sequential step limits the overall scalability of the application (Amdahl's Law).

- **Scheduling Overhead:** For the small workload, initialization and context switching overhead on 16 threads offset the performance gains of parallel execution.

4.5 Resource Footprint

During the evaluation run, memory usage peaked at:

Peak Memory Usage: 45.41 MB

Zero-Copy Memory Footprint Analysis:

The low memory usage is a direct result of Cassandra's zero-copy architecture:

1. **Streaming Tokens:** `lol_html` avoids loading the complete HTML structure into memory, keeping memory usage constant regardless of file size.
2. **Buffer Re-use:** Buffers are re-used across parsing stages, minimizing heap allocations.
3. **Reference Sharing:** Heavy data structures, such as domain blocklists and configurations, are shared across threads using `Arc<T>` pointers rather than being duplicated.

6. Critical Assessment

6.1 System Limitations

1. **Static Analysis Limitations:**
Cassandra relies on static parsing and blocklist validation. It cannot detect dynamic scripts constructed at runtime (e.g., using `eval` or string reconstruction).
2. **Sequential Logging Bottleneck:**
Writing the final JSON report sequentially using `serde_json::to_writer_pretty` introduces a performance bottleneck for large workloads, limiting parallel processing benefits.
3. **DNS Rebind Protection Gaps:**
Although the SSRF filter validates IP addresses during DNS resolution, it does not cache the resolved IP for the lifetime of the HTTP request, leaving the client vulnerable to DNS rebinding attacks if the remote server changes its resolution IP.
4. **Memory Bounds for Sub-Resources:**
While `total_bytes` and `total_requests` limit session footprint, incoming remote payloads are read into memory buffers before validation, exposing the system to potential memory exhaustion if several large files are fetched

concurrently.

6.2 Future Extensions

1. **Asynchronous JSON Streaming:**
Replace sequential report serialization with a streaming JSON writer to output events concurrently, eliminating the writing bottleneck.
2. **WebAssembly Script Sandboxing:**
Integrate a lightweight WebAssembly runtime (e.g., `wasmtime`) to execute and inspect scripts in a isolated sandbox environment, improving dynamic threat detection.
3. **SSRF Socket Binding:**
Improve the HTTP client to bind resolved and verified IP addresses to the socket connection, preventing DNS rebinding.
4. **Dynamic Policy Reloading:**
Use file watchers (e.g., `notify` crate) to monitor and reload TOML policies at runtime without restarting the main application.